

Instructor Analytics Dashboard

Spec synthesized from a grilling session pressure-testing the plan against this codebase's actual data and stack.

PROBLEM STATEMENT

Instructors have no way to see how their courses are actually performing from their dashboard. They cannot tell whether a course is selling, whether students are completing it, or where students disengage — they have to guess or ask support.

SOLUTION

Add an analytics view to the existing instructor dashboard (alongside the current per-student progress/quiz view in `instructor.$courseId.tsx`) showing, per instructor: revenue over time with a range toggle, a completion-rate KPI, an average-rating KPI, and — as a secondary, likely-deferred item — a lesson drop-off funnel. For instructors with more than one course, revenue is shown as a grid of small per-course charts rather than a single combined line, so one course's performance can't hide behind another's.

USER STORIES

1. As an instructor, I want to see my course's revenue over time, so that I know whether it's selling.
2. As an instructor, I want to toggle the revenue chart between 7d, 30d, and 90d ranges, so that I can look at short-term or longer-term performance.
3. As an instructor with a single course, I want a simple revenue chart with no aggregation overhead, so that the common case stays simple.
4. As an instructor with multiple courses, I want a per-course revenue breakdown (not one summed total), so that a declining course isn't hidden by a growing one.
5. As an instructor with many courses, I want the per-course breakdown to stay readable (small multiples grid, sorted by whoever needs attention most), so that it doesn't become an unreadable tangle of overlapping lines.

6. As an instructor, I want a total-revenue summary number separate from the per-course charts, so that I have one clear top-line fact without it inviting false trend-reading across mixed courses.
7. As an instructor, I want to see my course's completion rate, so that I know how many enrolled students actually finish.
8. As an instructor, I do not want my completion rate to look worse just because I recently ran a successful promotion and gained many new (not-yet-finished) students, so that the metric doesn't punish success.
9. As an instructor, I want to see my course's average rating and review count, so that I know how it's perceived.
10. As an instructor, I want the analytics view to visually match the rest of the dashboard (including dark mode), so that it doesn't look like a bolted-on feature.
11. As an instructor with a brand-new course, I want the revenue chart to show honestly sparse data rather than a fabricated trend line, so that I'm not misled by noise dressed up as a signal.
12. As a platform maintainer, I want the underlying revenue/completion/rating calculations to live in testable service functions, so that they can be verified without going through the UI.
13. (Secondary/deferred) As an instructor, I want to see where in my course students tend to stop progressing (a lesson drop-off funnel), so that I know which lesson to improve first.
14. (Secondary/deferred) As an instructor, I want the drop-off view to reflect true stopping points, not just the last lesson marked complete, so that the signal isn't cruder than it needs to be.

IMPLEMENTATION DECISIONS

- **Data sources, all already present:** `purchases` (revenue, timestamped per transaction), `enrollments` (`enrolledAt` / `completedAt`, for completion), `course_reviews` (1–5 rating). No schema changes needed for revenue, completion, or rating.
- **Charting library:** Recharts, used through shadcn's `chart` component wrapper (the codebase already uses shadcn/radix-ui throughout). The wrapper is a thin, copy-pasted layer over Recharts, not a separate dependency — it wires the app's existing CSS theme tokens (including dark mode) into the chart for no meaningful extra weight over using Recharts directly. Bare/unwrapped Recharts was considered and rejected: it would require manually re-wiring theming the wrapper already provides for free.

- **Per-course breakdown display:** small multiples (one compact chart per course, in a grid, sorted by whichever course most needs attention) — not a single summed line (which can mask a declining course behind a growing one) and not a single chart with all courses' lines overlaid (which stops being readable past a handful of courses).
- **Completion rate calculation:** must use a recency-aware denominator (e.g., only counting or cohorting students enrolled more than some minimum window, such as 30 days) rather than a flat `completed / total enrollments` ratio, which otherwise makes a successful recent promotion look like a performance decline.
- **No trend/seasonality modeling in v1:** revenue charts show the raw time series only. Actual purchase history in this environment is 6 transactions over 35 days, and structurally, every course starts with zero history regardless of platform maturity — there isn't enough data, ever, for a new course, to support statistical trend or seasonality claims at launch.
- **Lesson drop-off funnel is demoted to secondary/likely out of v1:** `lesson_progress` has no `startedAt`, only a completion status, and no existing query joins progress data against lesson ordering to compute "furthest lesson reached." `video_watch_events` has the right shape (timestamped, append-only) but nothing currently aggregates it into a funnel. If built, this needs new aggregation logic, not just a new chart.
- **Cross-course aggregation is a deliberate bet, not validated demand:** at present, 0 instructors in this environment have more than one course. Building the multi-course view now is an explicit growth bet rather than a response to observed usage.
- **Open technical item:** Recharts' `ResponsiveContainer` measures container width client-side; this needs to be verified against this app's React Router 7 SSR setup before commit, to rule out a 0-width flash on first paint.

TESTING DECISIONS

- Test external behavior of data-shaping functions (given rows in, correct aggregated numbers out), not Recharts rendering internals or implementation details of how a query is written.
- **Modules to test:** new analytics aggregation functions (revenue-by-day/range, completion rate with recency cutoff, average rating) — proposed to live alongside existing services as something like an `analyticsService.ts`.
- **Prior art in this codebase:** `purchaseService.ts`, `enrollmentService.ts`, and `progressService.ts` already follow a plain-function, service-layer pattern (e.g. `calculateProgress`, `getNextIncompleteLesson`) tested independently of any route/component. New analytics functions should follow the same pattern and same test style (vitest, per the project's existing `test` script) rather than introducing component/UI-level tests for chart rendering.

OUT OF SCOPE

- Explaining *why* revenue moved (no page-view/traffic or refund data exists in this schema to support that).
- Refund tracking.
- Conversion-funnel analytics (page views → purchase).
- Multi-currency handling (schema has a single integer price field, no currency field).
- Statistical trend or seasonality detection, for the reasons given above.

FURTHER NOTES

- The cross-course aggregation feature is being built ahead of actual demand (0 multi-course instructors today). If that segment doesn't materialize, this is the part of the feature most likely to go unused — worth revisiting before or shortly after launch.
- A concrete bundle-size budget for adding Recharts was never established; "don't want something heavy" was raised as a concern but not quantified.
- Whether the drop-off funnel ships at all in v1 was left undecided during the discussion — treat it as cut unless explicitly greenlit, given the data-model gap described above.